Synopsis on

AGENT RELIABILITY LAB

Controlled Fault Injection and Evidence-Based Failure Analysis for AI Agents

in partial fulfillment of the requirement for the degree

of

Bachelor of Technology In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Suryansh Singh (2301330100208)

Simran Kumari (2301330100200)

Tanishka Sahu (2301330100210)

Group: CSE/CSPROJECT/2027/G053

Under the supervision of

Mr. Pradeep Kumar (Asst. Professor, CSE)

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY

GREATER NOIDA

(An Autonomous Institute)

Department of Computer Science and Engineering

Session 2026-27

Index

Sr. No.	Topics	Page No.
1	Introduction	1
1.1	Background and Motivation	1
1.2	Field of the Project and Technology Used	1
1.3	Important Technical Terms	2
1.4	Objectives and Scope	2
2	Existing Systems	4
2.1	Literature and Tool Survey (Tabular)	4
2.2	Comparative Discussion and Research Gaps	4
3	Problem Statement	6
4	Proposed Methodology	7
4.1	System Architecture	7
4.2	Overview of Modules	8
4.3	Use Cases	8
4.4	Entity-Relationship Model	9
4.5	Data Flow Diagrams	10
4.6	Flowcharts of Core Processes	11
4.7	Core Algorithms and Implementation Plan	14
5	Feasibility Study	16
6	Facilities Required for Proposed Work	18
7	Conclusion	19
8	References	20

Supervisor Sign: _____

1. Introduction

1.1 Background and Motivation

Agents built on large language models (LLMs) no longer answer a single question. They plan, call tools such as search engines, databases, and calculators, read what comes back, recover from errors, and continue for many steps. This shift has created a practical reliability problem. An agent may finish a task in one attempt and fail the same task in the next. It may skip a tool it needed, call a tool it was never given, repeat a call after the answer arrived, or give up after a single timeout. Most benchmarks still report one number, the share of tasks finished, measured once per task in a clean environment where nothing breaks.

That number hides three things a team needs before it trusts an agent: why runs fail, how consistently the agent succeeds, and what its reliability costs in money and time. Agent Reliability Lab is proposed as a crash-test laboratory for AI agents. It breaks tools on purpose in a repeatable way, runs every agent several times on the same tasks, explains each failed run with evidence, and selects the cheapest configuration that stays reliable. It does not train a new model; it provides the experiment around the model, so that two agents can be compared fairly and the difference can be explained.

1.2 Field of the Project and Technology Used

The project belongs to Computer Science and Engineering, in the area of agentic AI, evaluation of LLM systems, and software reliability engineering. It borrows two ideas from outside machine learning: fault injection from chaos engineering, where failures are introduced deliberately to test resilience, and paired experimental design from statistics, where two treatments are compared on the same subjects.

The implementation is a Python 3.11+ package named `relai` with a command-line interface. Pydantic models describe tasks and

trajectories, task suites are YAML files, results are stored in SQLite, and reports are rendered as HTML. Testing uses pytest and linting uses Ruff. Live models connect through any OpenAI-compatible API, including local models served by Ollama.

1.3 Important Technical Terms

- **Agent:** a program that works on a task by planning and calling tools; here, anything with a name and a `run(task)` method.
- **Trajectory:** the ordered list of steps in one run: plan, tool call, observation, error, recovery, and final answer.
- **Trial:** one attempt at a task; every task is attempted several times.
- **Fault injection:** making tool calls time out or fail on purpose, on a seeded schedule.
- **pass@k and pass^k:** the chance that at least one, or all, of k attempts succeed.
- **Failure mode:** a diagnosed cause of failure: planning, tool misuse, context, recovery, or horizon.
- **Unclassified:** a failed run that no rule explains, kept separate instead of being forced into a mode.
- **Paired test:** a significance test (McNemar) that compares two agents on the same task and trial.
- **Pareto front:** the settings that cannot be improved on success without costing more money or time.

1.4 Objectives and Scope

The objectives of the proposed work are:

1. To build an evaluation framework that measures success, correctness, cost, latency, tool use, recovery, step use, and reliability across repeated trials.
2. To develop automated failure analysis and recovery evaluation, using seeded fault injection and an evidence-based failure taxonomy.

3. To enable controlled experiments and optimization: fair comparison of models, prompts, and policies, and selection of the cheapest setting that meets a reliability target.

The scope is a local, offline-capable laboratory that any agent can plug into. A hosted monitoring service, model training, and a graphical dashboard are outside the scope of the first version and are planned as later extensions.

2. Existing Systems

Evaluation of LLMs and agents is an active field. Some systems measure models on static prompts, some score agents in task environments, some record traces for debugging, and recent work studies reliability and fault injection directly. Table 2.1 places the most relevant approaches side by side.

2.1 Literature and Tool Survey

Table 2.1: Survey of existing evaluation approaches

S. No.	System / Work	Primary focus	Limitation for this problem
1	HELM; LM Evaluation Harness	Broad benchmarking of models on static datasets	Scores single responses, not multi-step agent trajectories
2	AgentBench, GAIA, SWE-bench, WebArena	Agent tasks and environments with task success	Report success once per task; no failure diagnosis
3	tau-bench (2024)	Tool-agent-user tasks; introduced pass^k	No injected faults or per-run diagnosis
4	HAL leaderboard (ICLR 2026)	Cost-aware evaluation of agents at scale	Benchmark infrastructure; no shared fault schedule
5	Agent reliability metrics (ICML 2026)	Twelve metrics: consistency, robustness, predictability, safety	Metrics on benchmarks; no paired test or self-checked classifier
6	ToolMisuseBench, ToolBench-X (2026)	Replayable tool faults and hazards	Benchmark datasets; faults not used as a paired factor
7	AgentChaos (2026)	Chaos engineering through an LLM API proxy	Injects at model layer; no taxonomy of agent failures
8	MAST, TRAIL, Who&When (2025)	Failure taxonomies and attribution in traces	Human or LLM labels; best step attribution only 14.2%
9	LangSmith, Langfuse, Phoenix	Production tracing and scoring	Observe live traffic; no controlled experiments
10	Inspect AI, DeepEval, Promptfoo	Evaluation frameworks and CI tests	General scoring; no agent fault schedule or recovery metric

2.2 Comparative Discussion and Research Gaps

Model benchmarks answer whether a model can solve a prompt. Agent benchmarks add environments, but their contribution is the task, not an analysis layer. Tracing platforms capture rich logs and are the right choice for production debugging, but they observe traffic rather than run controlled experiments. Recent reliability and fault-injection research is the closest to this project; it proposes metrics and

benchmarks, while this project provides a small laboratory in which any team can run the experiment on its own agent.

The survey shows five gaps that this project addresses:

- **Success-only reporting:** one score hides why a run failed and how consistent the agent is.
- **Unfair comparison:** when failures are random per run, two agents never meet the same conditions.
- **Missing recovery accounting:** retry quality after timeouts is rarely measured.
- **Unverified diagnosis:** automated failure labels are seldom checked against known ground truth.
- **Cost-blind selection:** reliability is reported without the cost and latency it requires.

3. Problem Statement

Background. Tool-using AI agents fail in ways that a single accuracy number cannot explain. One agent guesses without calling a required tool, another calls a forbidden tool, a third repeats a call it has already answered, and a fourth abandons a task after one timeout. Existing benchmarks publish task success measured once in a clean environment, and existing tracers publish logs. Neither lets a team run a fair, repeatable experiment in which agents meet the same failures, every failure is explained, and the cost of reliability is visible.

Problem. Design and implement Agent Reliability Lab, a framework that runs any agent on a shared task suite for several trials under seeded tool faults that are identical for every agent; records each trajectory; measures success, pass^k, consistency, cost, latency, recovery, and step use with confidence intervals; classifies planning, tool-misuse, context, recovery, and horizon failures with evidence while leaving unexplained failures unclassified; tests whether two agents differ significantly; and recommends the cheapest configuration that meets a reliability target.

Expected resolution. A small agent interface, analysis that only reads stored trajectories, a fault schedule independent of the agent, rules pinned by tests, and a self-check that proves the classifier finds defects planted on purpose.

4. Proposed Methodology

The methodology is organized as modules that follow the data. Domain types come first because every later pass reads the same objects. The agent interface and the tool environment come next, so that offline policies and live models share one seam. The harness, metrics, failure analysis, statistics, and optimizer then operate on recorded trajectories without calling agents again.

4.1 System Architecture

Fig. 4.1 shows four layers. Inputs are a task suite and an agent. The execution layer runs trials through a fault-injecting environment. The analysis layer reads trajectories only. The output layer compares, optimizes, stores, and reports.

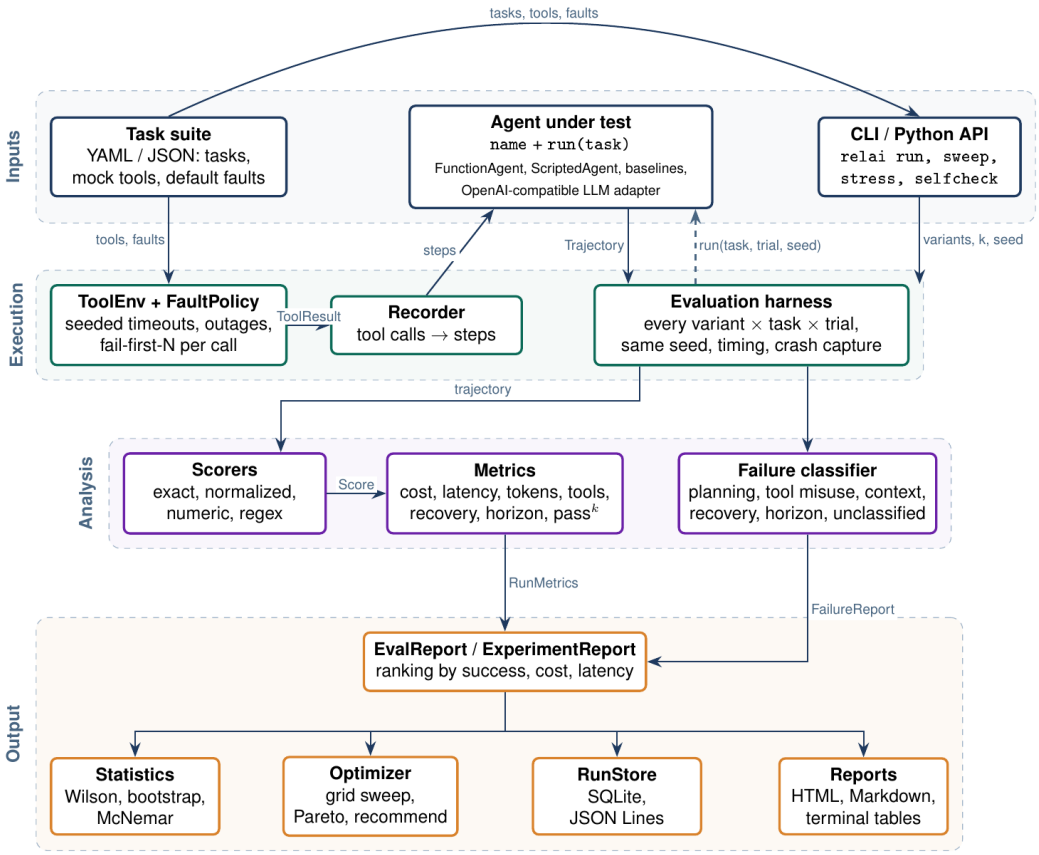


Figure 4.1: System architecture of Agent Reliability Lab

4.2 Overview of Modules

Table 4.1: Modules of the proposed system

Module	Functionality
Domain types and agent seam	Task, Step, Trajectory, Score, RunMetrics, FailureReport; any agent with name and run(task)
Tool environment	Mock tools, seeded fault policy (timeouts, outages, fail-first), recorder of steps
Evaluation harness	Every variant on every task for k trials; timing; crash capture; pass@k, pass^k, consistency
Metrics and scorers	Exact, normalized, numeric, regex scorers; cost, latency, tokens, recovery, steps
Failure analysis	Five failure modes with evidence, root cause, unclassified outcome
Statistics	Wilson and bootstrap intervals; exact McNemar paired test
Optimizer	Grid sweep, Pareto front, cheapest setting meeting a target
Suites and baselines	YAML task suites; reference agents with planted defects; classifier self-check
Store, reports, CLI	SQLite and JSON Lines; HTML and Markdown reports; relai commands; LLM adapter

4.3 Use Cases

The primary actor is a researcher or developer who defines suites, configures faults, plugs in agents, runs evaluations, compares variants, and sweeps settings. The agent under test is a secondary actor that only produces trajectories; a model API is used by live agents (Fig. 4.2).

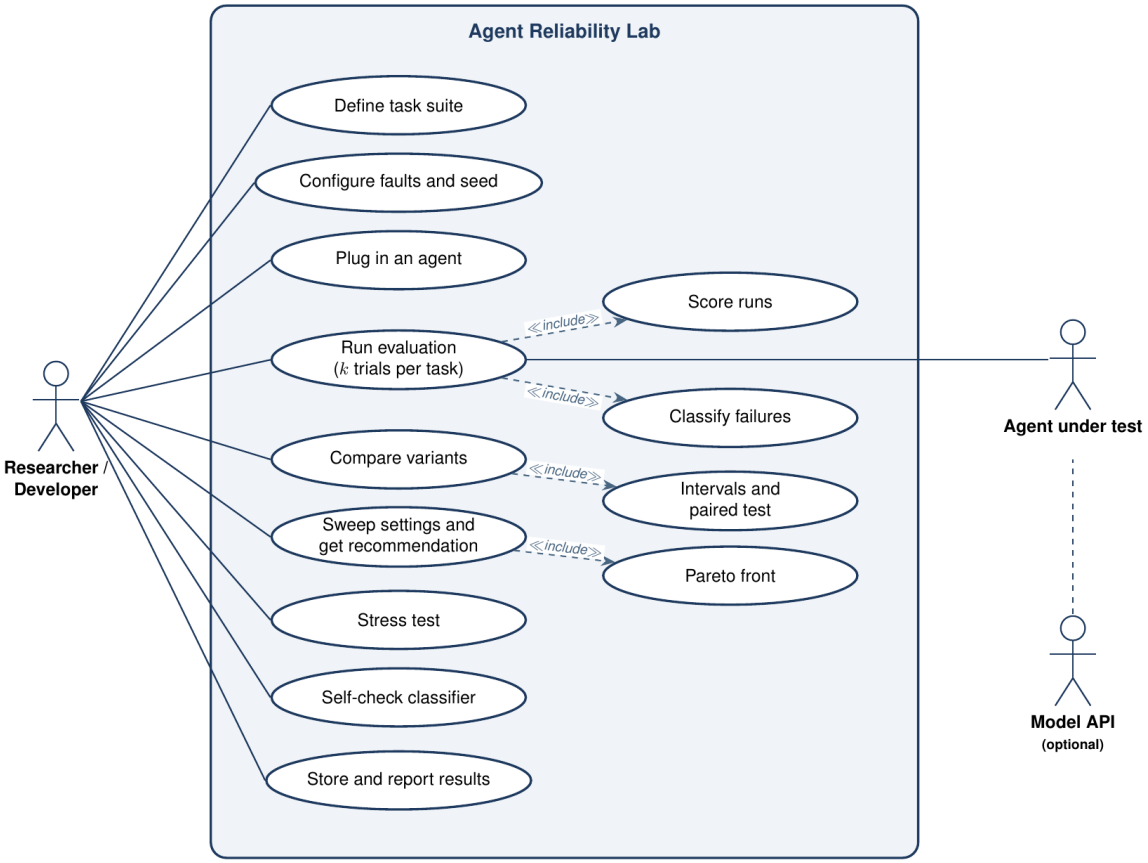


Figure 4.2: Use case diagram

4.4 Entity-Relationship Model

Results persist in a database whose entities mirror the in-memory reports. An experiment contains variants, a variant produces runs, and each run records steps and diagnosed failure modes (Fig. 4.3). Each run also keeps its complete record as JSON, so stored experiments reload without loss.

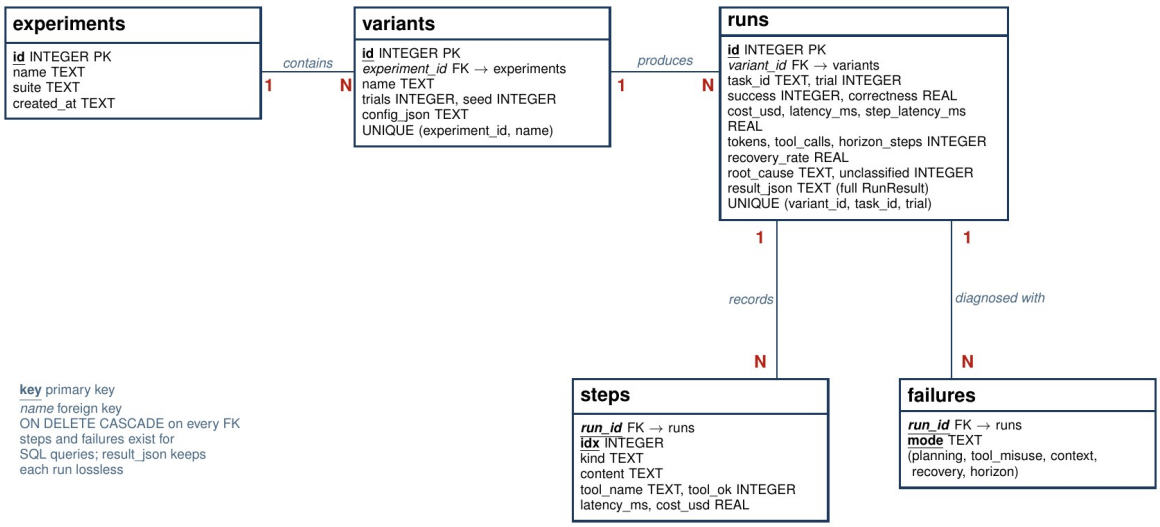


Figure 4.3: Entity-relationship model of the run store

4.5 Data Flow Diagrams

The context diagram (Fig. 4.4) shows the researcher, the agent under test, and an optional model API around the laboratory. The level 1 diagram (Fig. 4.5) divides the laboratory into six processes and four data stores.

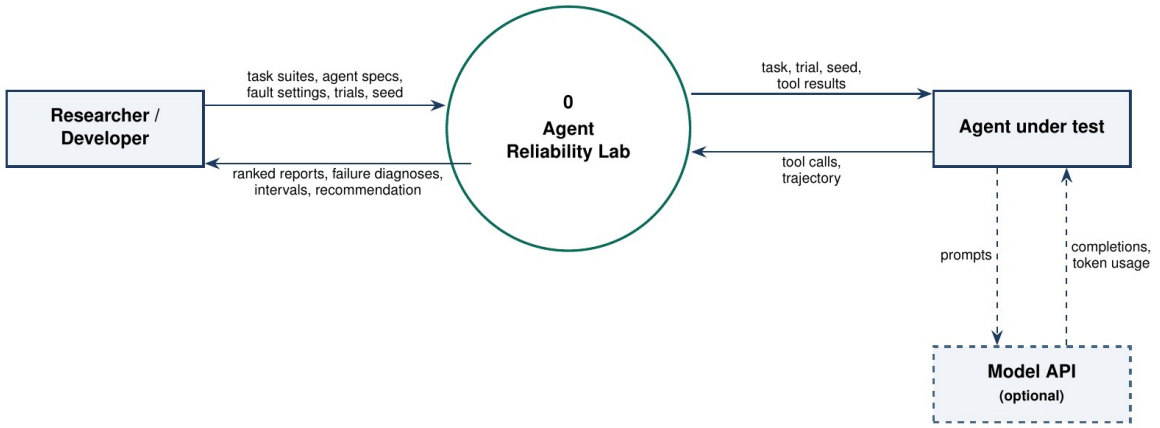


Figure 4.4: Level 0 data flow diagram

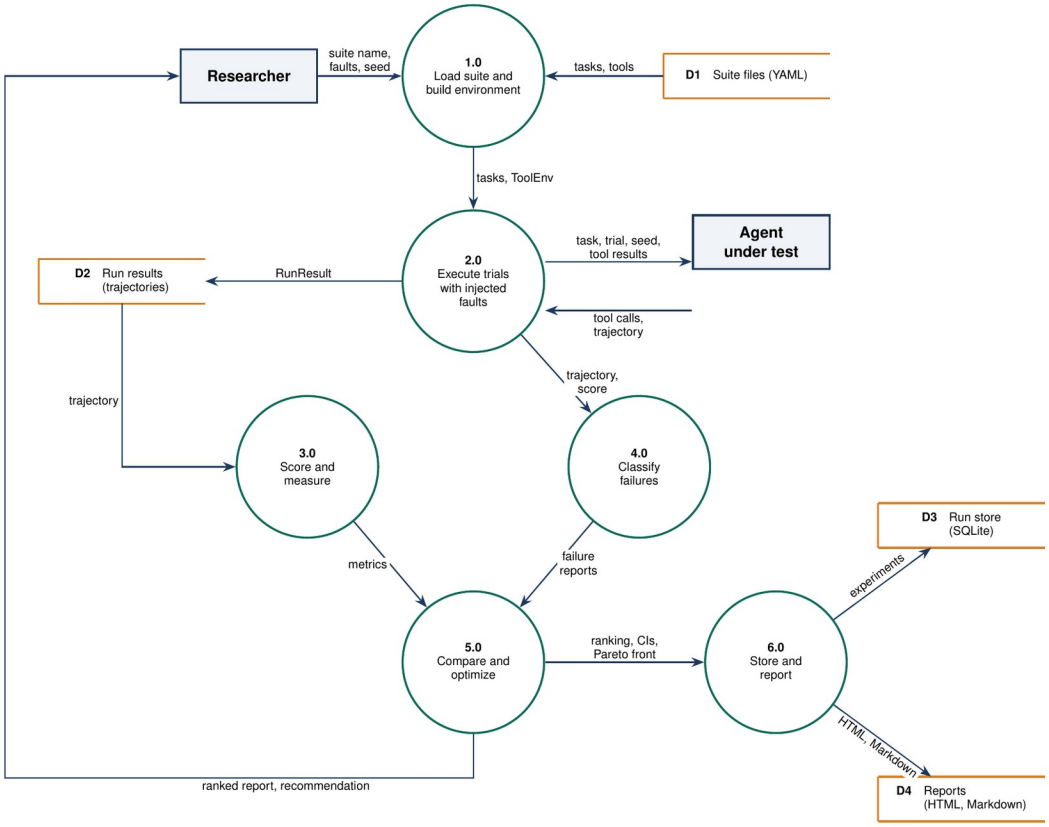


Figure 4.5: Level 1 data flow diagram

4.6 Flowcharts of Core Processes

Fig. 4.6 is the classifier: five detectors run, the earliest evidence becomes the root cause, and an unexplained failure stays unclassified. Fig. 4.7 is the evaluation loop: for every variant, task, and trial the harness times the agent, records crashes as failed runs, and scores, measures, and classifies each trajectory. Fig. 4.8 is the configuration search.

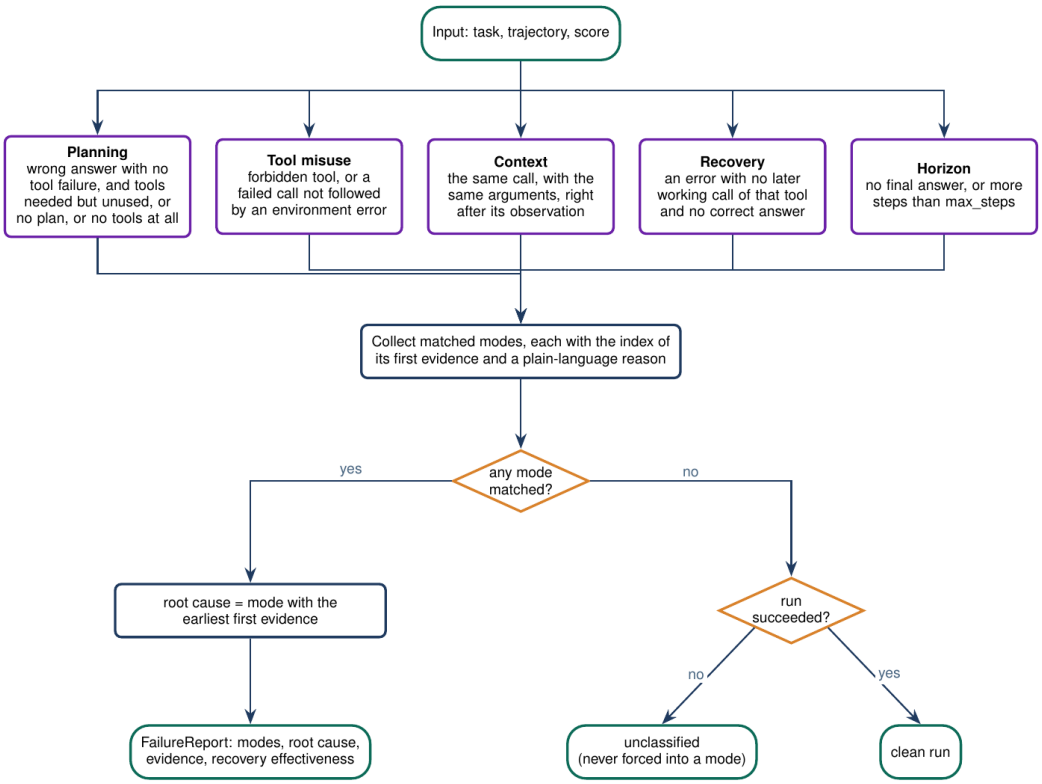


Figure 4.6: Flowchart of failure classification

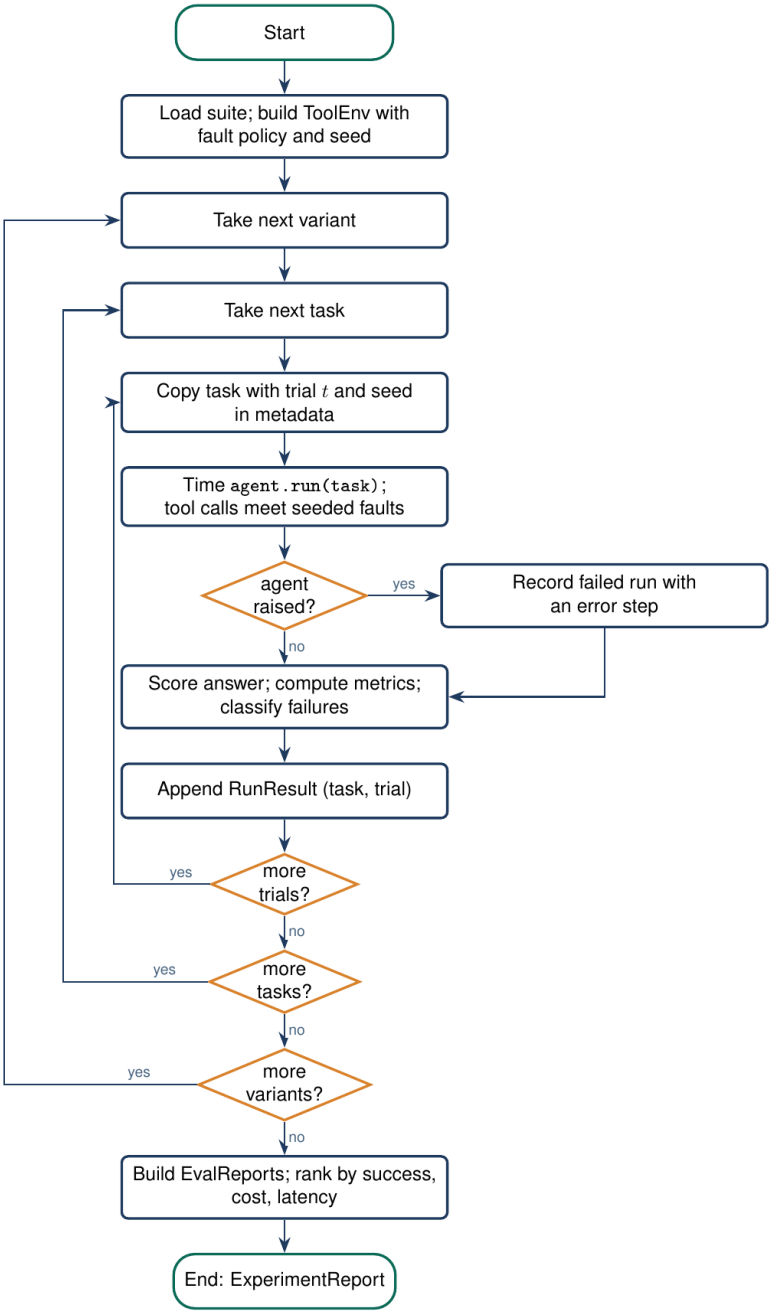


Figure 4.7: Flowchart of the evaluation harness

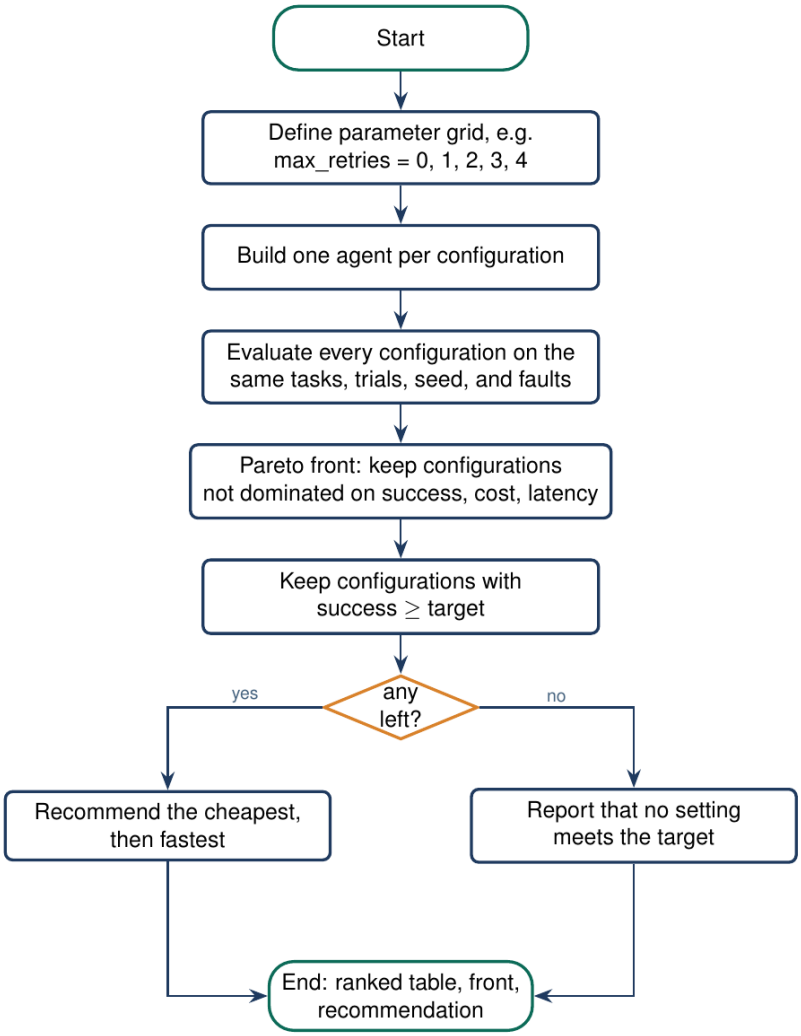


Figure 4.8: Flowchart of configuration search

4.7 Core Algorithms and Implementation Plan

- **Seeded fault schedule:** the n-th call to a tool in trial t of a task draws u = h(seed, task, t, tool, n); the call times out when u is below the timeout rate. The draw never depends on the agent, so every agent meets the same faults.
- **Reliability estimators:** with c successes in n trials, pass@k = 1 - C(n-c, k) / C(n, k) and pass^k = C(c, k) / C(n, k).
- **Paired comparison:** runs of two agents are paired by task and trial, and the exact McNemar test uses the pairs where only one agent succeeded.
- **Selection:** the Pareto front over success, cost, and latency, then the cheapest setting whose success meets the target.

Development follows a test-first plan: domain types and the agent seam, then metrics and the failure rules with fixtures, then the fault environment and trials, then statistics, optimization, storage, reports, and the command-line interface (Fig. 4.9).

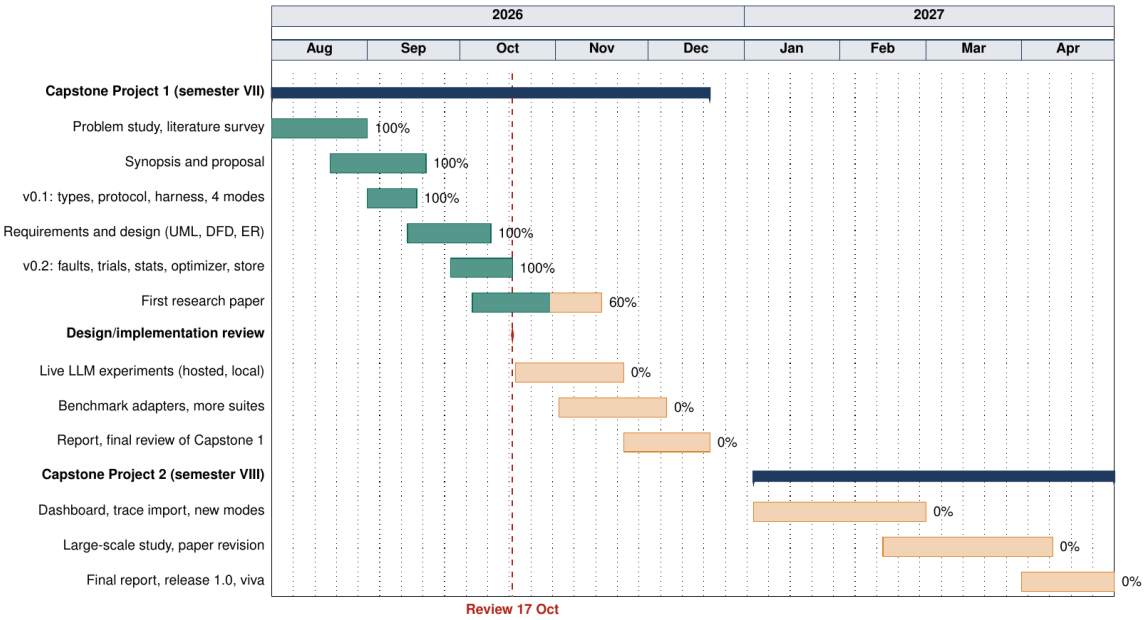


Figure 4.9: Project schedule (Gantt chart)

5. Feasibility Study

Feasibility is the first step of software engineering: whether the laboratory is needed, whether it can be built with the available skills and tools, and whether it can be completed in the project calendar.

5.1 Technical Feasibility

The stack is mature and already taught in the department: Python, Pydantic, pytest, SQLite, and YAML. No GPU, distributed system, or paid service is required. Offline reference agents remove the dependence on model APIs during development, and the failure rules are deterministic predicates over steps that are easy to test. A working prototype already runs about 6,500 scored runs per second on a laptop.

5.2 Operational Feasibility

Users install the package in a virtual environment and run one command per experiment. Results appear as terminal tables, JSON, or a standalone HTML report that supervisors can read without extra software. Later batches can plug in their own agents through the same interface.

5.3 Economic Feasibility

All tools are free and open source, and the package is MIT licensed. Offline experiments cost nothing. When live models are used, cost is itself a measured quantity, so expensive configurations are visible, and local models through Ollama keep cost at zero.

5.4 Schedule Feasibility

Every module has a testable exit criterion, and work can proceed without model access because analysis never calls agents. The main risk is scope growth; it is controlled by the rule that a new failure mode is added only with a test, and that agents attach through one interface.

5.5 Need and Significance

Agents are entering products, internships, and coursework, yet teams still compare them by anecdote or by a single success rate. A shared laboratory makes those comparisons fair, repeatable, and explainable. For the department it is a reusable evaluation asset; for students it is practice in experimental design, metric definition, and evidence-based failure analysis.

6. Facilities Required for Proposed Work

The work is software-intensive and needs no special laboratory instrument.

Table 6.1: Facilities required

Category	Item	Purpose
Hardware	Laptop or lab PC, 8 GB RAM, 20 GB free disk, CPU only	Development, tests, demonstration
Operating system	Linux, macOS, or Windows	Any platform with Python
Language	Python 3.11 or newer, venv, pip	Implementation
Libraries	Pydantic 2, PyYAML	Typed models, task suites
Development	pytest, Ruff, Git, GitHub Actions	Tests, lint, version control, CI
Documentation	LaTeX, Word or LibreOffice	Synopsis, report, paper
Optional	OpenAI-compatible API key or Ollama	Live LLM experiments
Data	Built-in task suites	Repeatable experiments without downloads

A GPU is not required. A machine with more memory helps only if a large local model is served with Ollama. Internet access is needed for installing packages and for hosted models, not for offline experiments.

Human resources and time. The work is carried out by a team of three students under the supervision of a faculty member, over two semesters: the design and core implementation in semester VII and the live model study, extensions, and final report in semester VIII. Institute laboratory PCs are sufficient for the viva demonstration.

7. Conclusion

This synopsis proposes Agent Reliability Lab, a crash-test laboratory for AI agents, as a final-year project of the Department of Computer Science and Engineering, Noida Institute of Engineering and Technology. The work addresses a concrete gap: teams can see whether an agent finished a task, but rarely why it failed, how consistently it succeeds, or what its reliability costs.

The proposed method keeps the agent interface small, injects tool faults on a seeded schedule shared by all variants, runs repeated trials, derives metrics only from trajectories, explains failures with evidence while leaving unknown failures unclassified, compares variants with paired tests, and recommends the cheapest reliable setting. Use cases, the entity-relationship model, data flow diagrams, and flowcharts show how the parts connect, and the feasibility study is favorable on every axis.

A working implementation already validates the design: with reference agents, the measured effect of faults matches its closed-form expectation and the classifier recovers every planted defect. The next phase evaluates live LLM agents and adds more suites and fault kinds. The outcome is not another chatbot but a reliability instrument that later batches can extend.

The expected outcomes of the completed project are:

- An installable, tested Python package and a failure taxonomy validated with planted defects.
- A walkthrough website that explains each feature in plain language, with definitions and worked commands.
- Reproducible experiments comparing live LLM agents under identical faults, and a research paper.

8. References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models", ICLR, 2023.

[2] X. Liu et al., "AgentBench: Evaluating LLMs as Agents", ICLR, 2024.

[3] G. Mialon et al., "GAIA: A Benchmark for General AI Assistants", ICLR, 2024.

[4] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?", ICLR, 2024.

[5] S. Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents", ICLR, 2024.

[6] S. Yao, N. Shinn, P. Razavi, K. Narasimhan, "tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains", arXiv:2406.12045, 2024.

[7] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, A. Narayanan, "AI Agents That Matter", arXiv:2407.01502, 2024.

[8] S. Kapoor et al., "Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation", ICLR, 2026.

[9] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, A. Narayanan, "Towards a Science of AI Agent Reliability", ICML, PMLR 306, 2026.

[10] "ToolMisuseBench: An Offline Deterministic Benchmark for Tool Misuse and Recovery in Agentic Systems", arXiv:2604.01508, 2026.

[11] "Beyond Function Calling: Benchmarking Tool-Using Agents under Tool-Environment Unreliability", arXiv:2606.25819, 2026.

[12] "AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection", arXiv:2608.06790, 2026.

[13] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?", arXiv:2503.13657, 2025.

[14] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, R. Qian, "TRAIL: Trace Reasoning and Agentic Issue Localization", arXiv:2505.08638, 2025.

[15] S. Zhang et al., "Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems", ICML, PMLR 267, 2025.

[16] P. Liang et al., "Holistic Evaluation of Language Models", Transactions on Machine Learning Research, 2023.

[17] UK AI Security Institute, "Inspect: A Framework for Large Language Model Evaluations", 2024.

[18] M. Chen et al., "Evaluating Large Language Models Trained on Code", arXiv:2107.03374, 2021.

[19] A. Basiri et al., "Chaos Engineering", IEEE Software, vol. 33, no. 3, pp. 35-41, 2016.

[20] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages", Psychometrika, vol. 12, no. 2, pp. 153-157, 1947.

[21] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference", Journal of the American Statistical Association, vol. 22, no. 158, pp. 209-212, 1927.

[22] R. S. Pressman, B. R. Maxim, "Software Engineering: A Practitioner's Approach", 9th ed., McGraw-Hill, 2020.